Virtual Memory and Caching

CS 2410

Wonsun Ahn

University of Pittsburgh

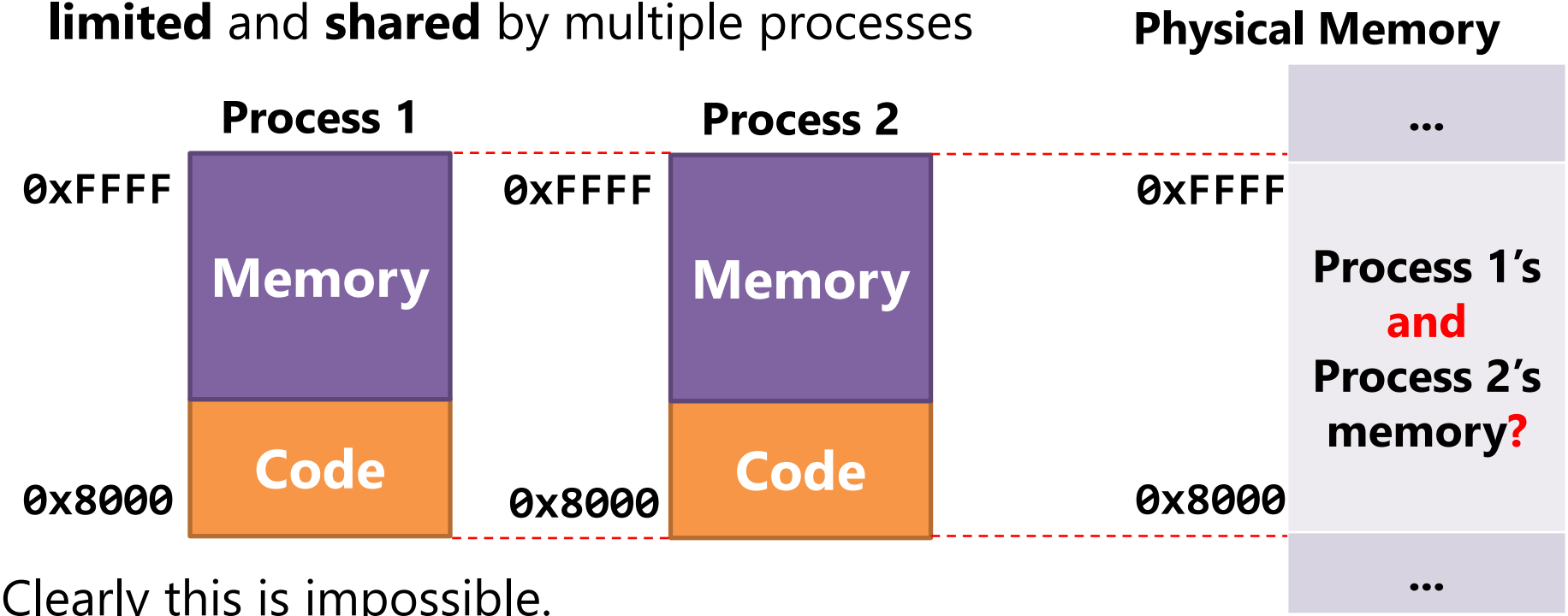
Virtual Memory and Caching

- So what does virtual memory have to do with caching?
- *A lot* actually.
- But first let's do a quick review of virtual memory

Virtual Memory Review

Virtual Memory: Type of Virtualization

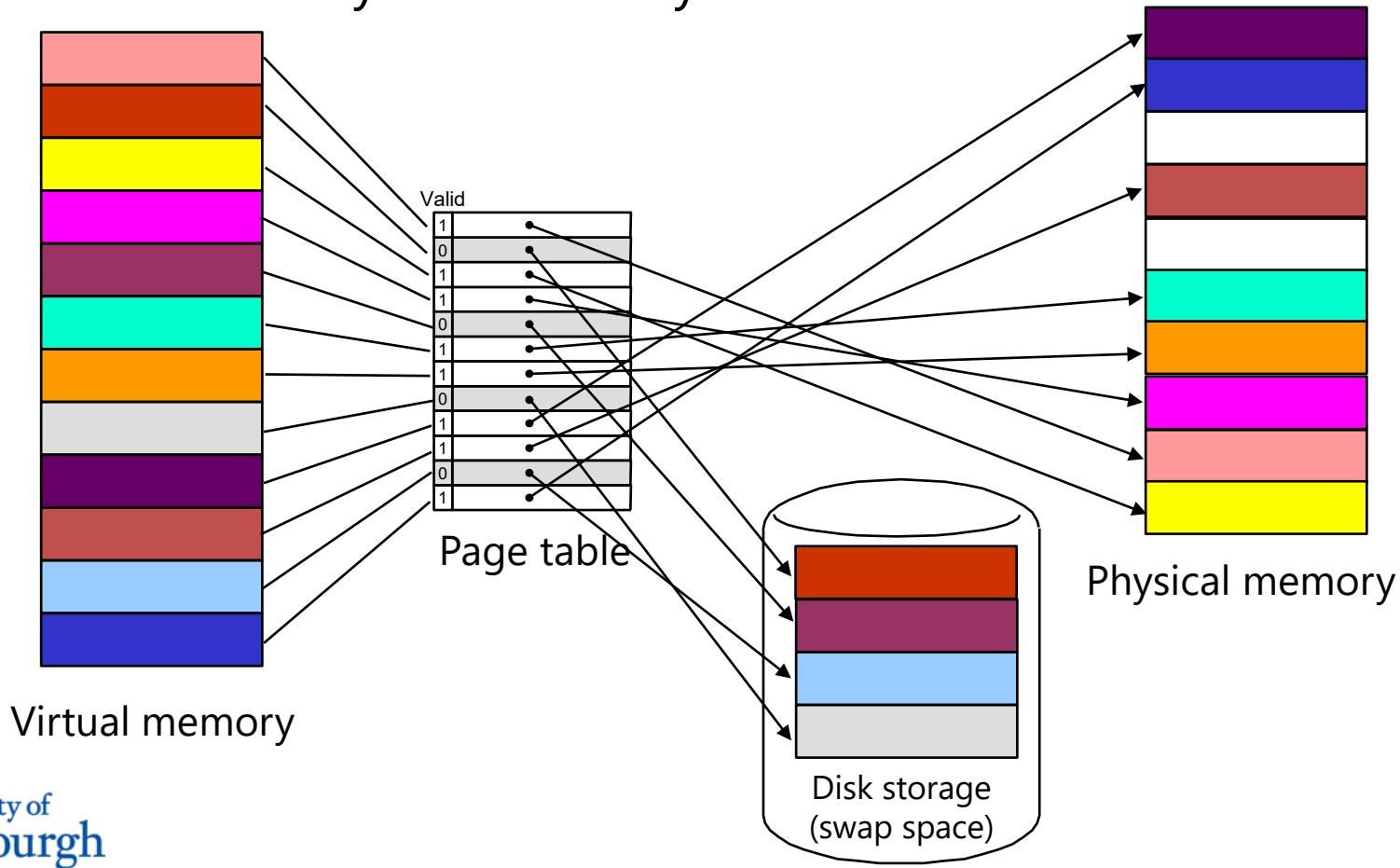
- **Virtualization**: hiding the complexities of hardware to software
- **Virtual Memory**: hides the fact that physical memory (DRAM) is **limited** and **shared** by multiple processes



Clearly this is impossible.
But programs see this view of memory.

Virtual Memory: Behind the Scenes

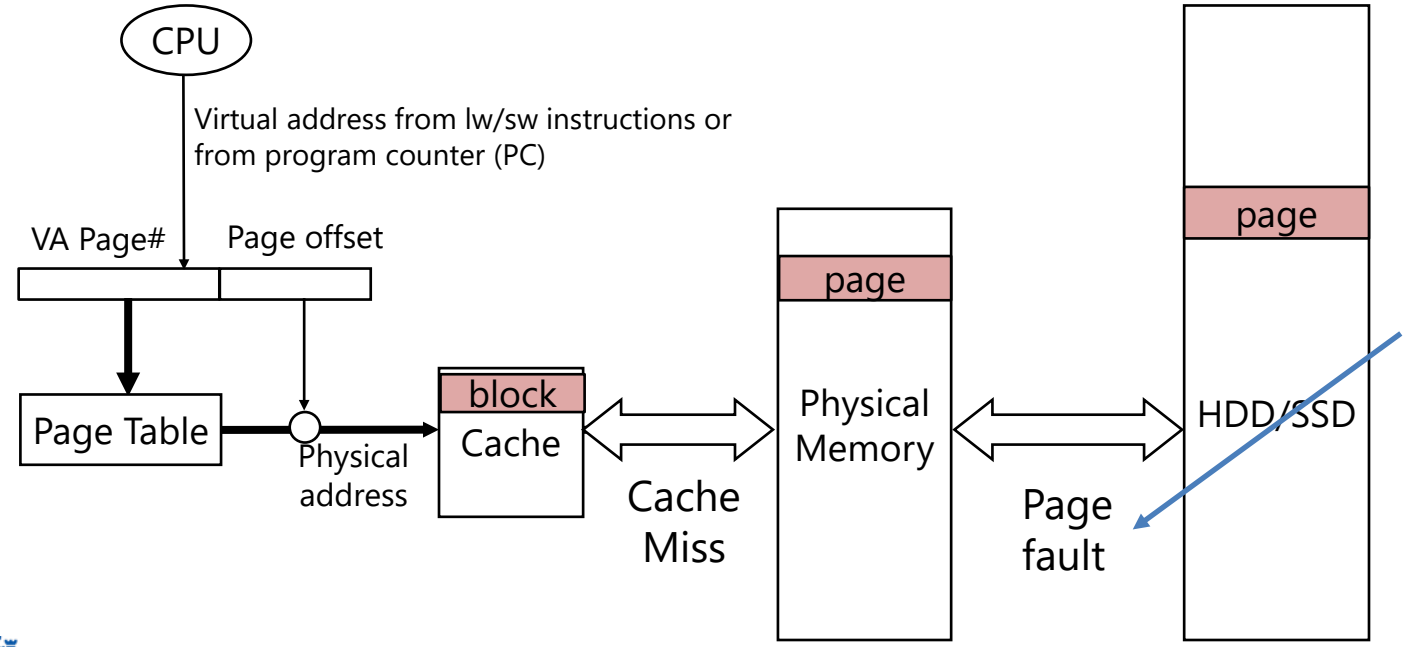
- **Pages** of memory are mapped to either physical memory or disk
  - Look familiar? Physical memory acts as a **cache** for disk storage



DRAM as Cache

DRAM is a cache for HDD/SSD

- Relationship between DRAM ↔ Disk is same as Cache ↔ DRAM
  - DRAM has **smaller capacity**: by **~100x** to **~1000x**
  - DRAM has **higher bandwidth**: by **~50x** for SSD, **~500x** for HDD
  - DRAM has **lower latency**: by **~1000x** for SSD, **~100000x** for HDD



Why called "fault"?

Causes a fault or exception in OS.
→ Triggers OS page fault handler that moves page to DRAM

What are the cache design parameters for DRAM?

- Block size?
  - **4 KB pages** → to better leverage **spatial locality**
  - Leads to pollution but DRAM has larger capacity than CPU cache

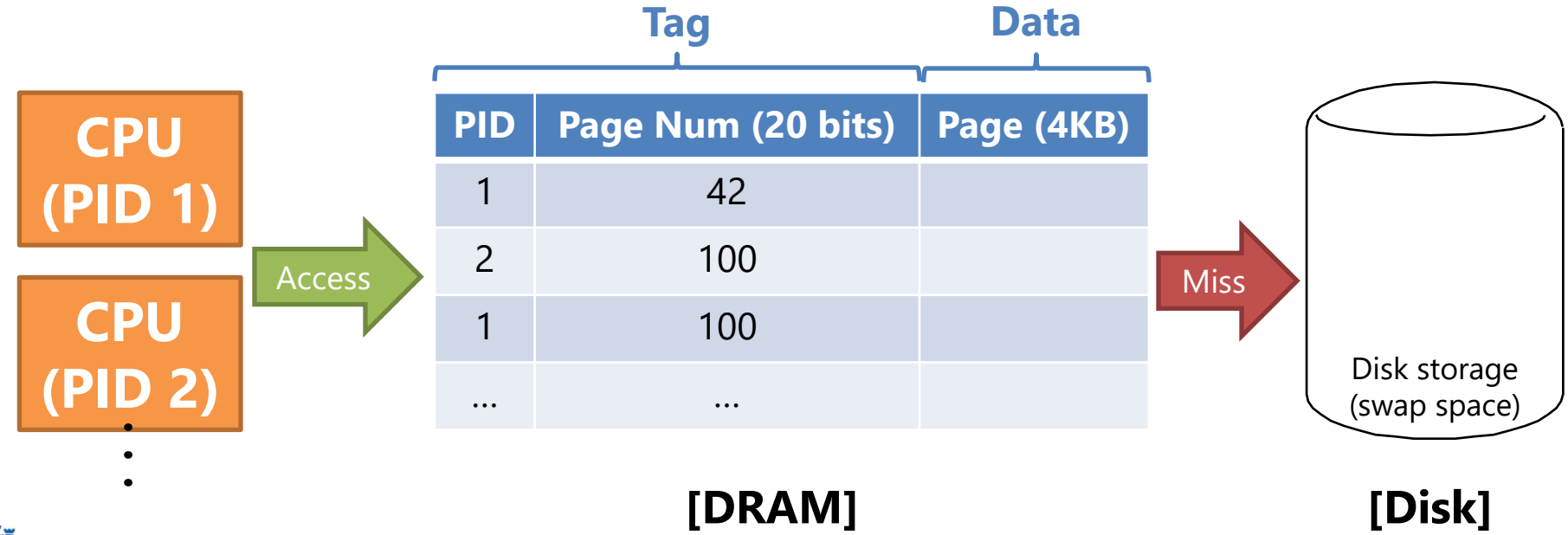
- Replacement scheme?
  - **LRU** → to better leverage **temporal locality**

- Write policy?
  - **Write-back** → to save **bandwidth**
  - Write through didn't make sense for Cache ⇔ DRAM memory bus
  - Write through for DRAM ⇔ Disk PCIe bus is just ludicrous

- Associativity?
  - **Fully-associative** → to save **bandwidth + latency**
  - A **page fault** is really really expensive in terms of both
  - But now **page hits become expensive** due to **lookup** cost

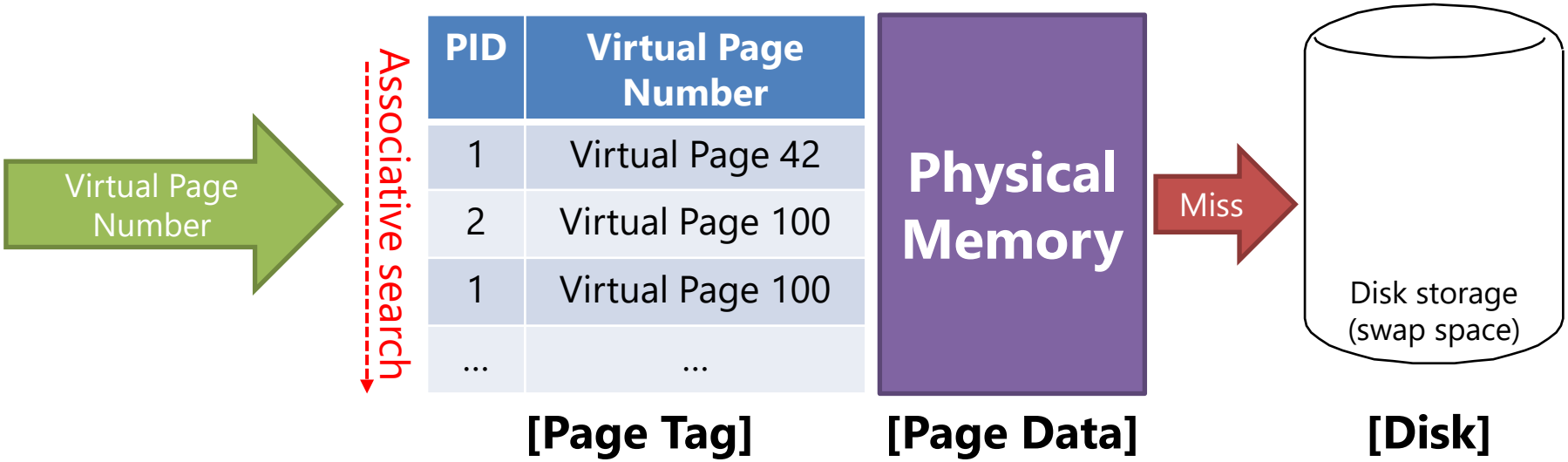
Physical Memory as a Cache

- If you treated each page as a cache block, what would be the tag?
  - 32-bit address: Page number (20 bits) | Page offset (12 bits)
  - Fully-associative: no index bits, 12 bits for offset, 20 bits for tag
- Tag must also include PID (Process ID) for the page
  - So that processes do not access each others' data!



Inverted Page Table: indexed by physical page number

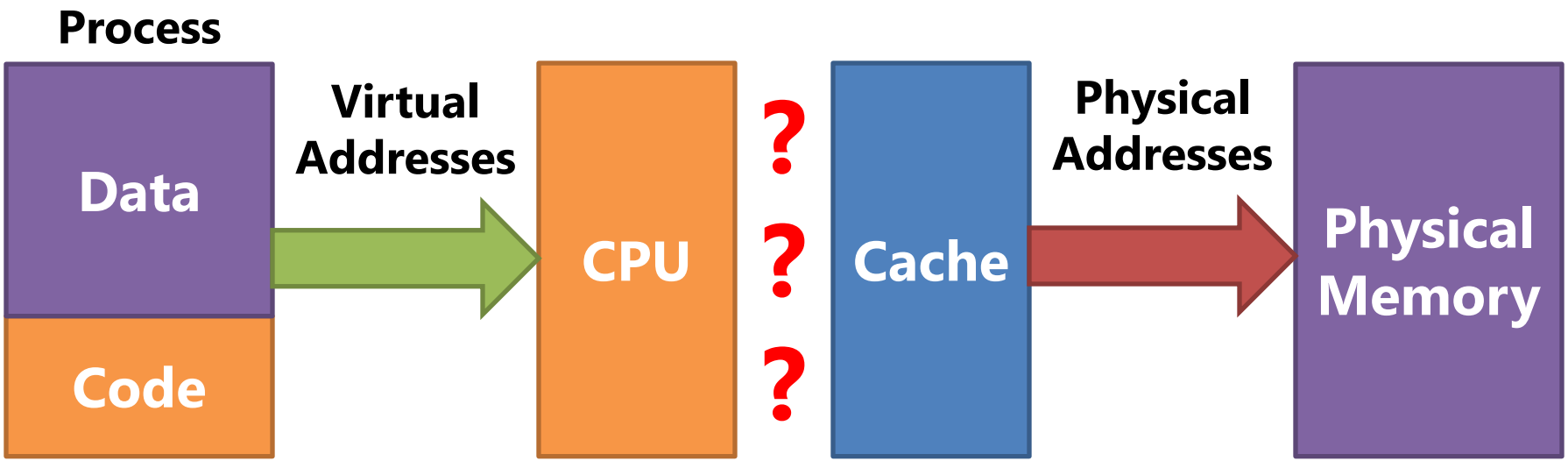
- This type of page table is called an **inverted page table**.



- Called inverted because table is indexed by physical page number
- **Pro**: Page table only as big as physical mem (**low space complexity**)
- **Con**: Associative search of page table (**high time complexity**)
  → Uses hash tables but with conflict chaining, lookup still costly

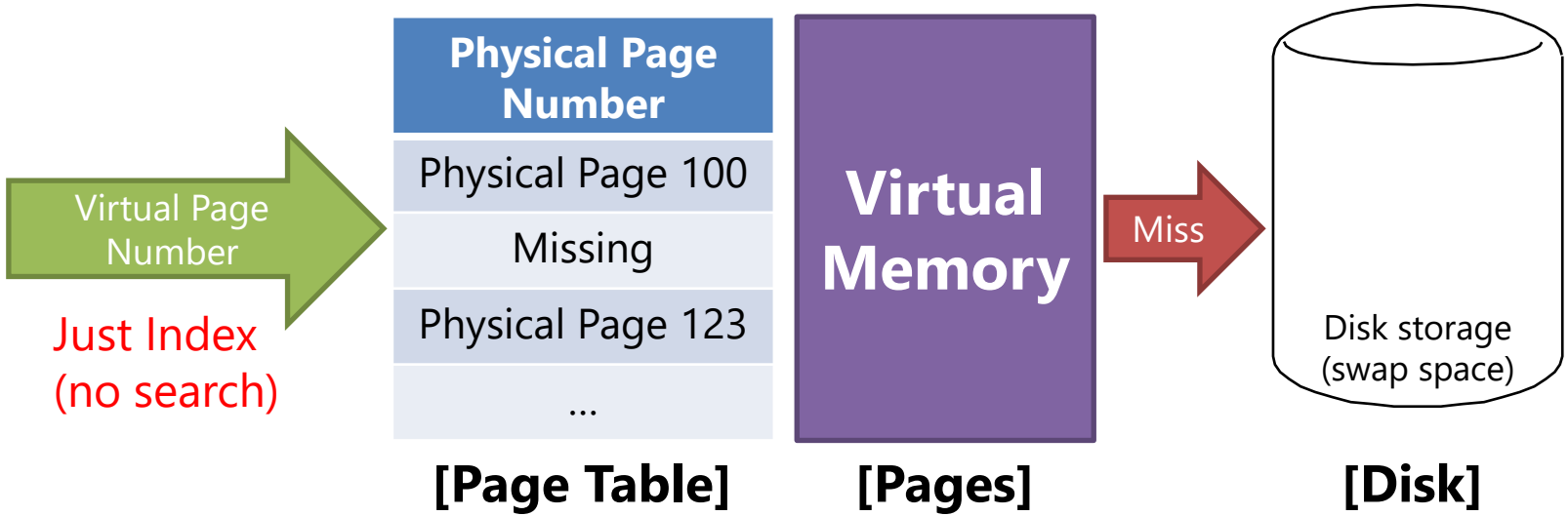
Page table lookups need to be just a few cycles long

- Programs use **virtual addresses** to refer to code and data
- DRAM and Caches use **physical addresses**
- How often does the conversion need to happen?
  - At MEM stage of every `lw` or `sw` instruction to convert data addr
  - At FETCH stage of every instruction to convert PC



Regular Page Table: indexed by virtual page number

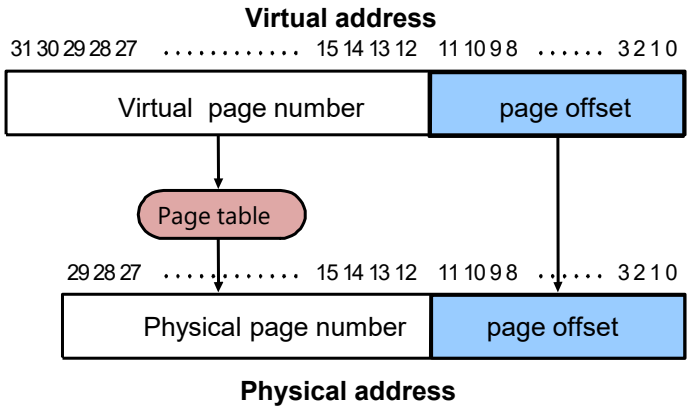
- Lookup is done by **indexing** page table using virtual page number.

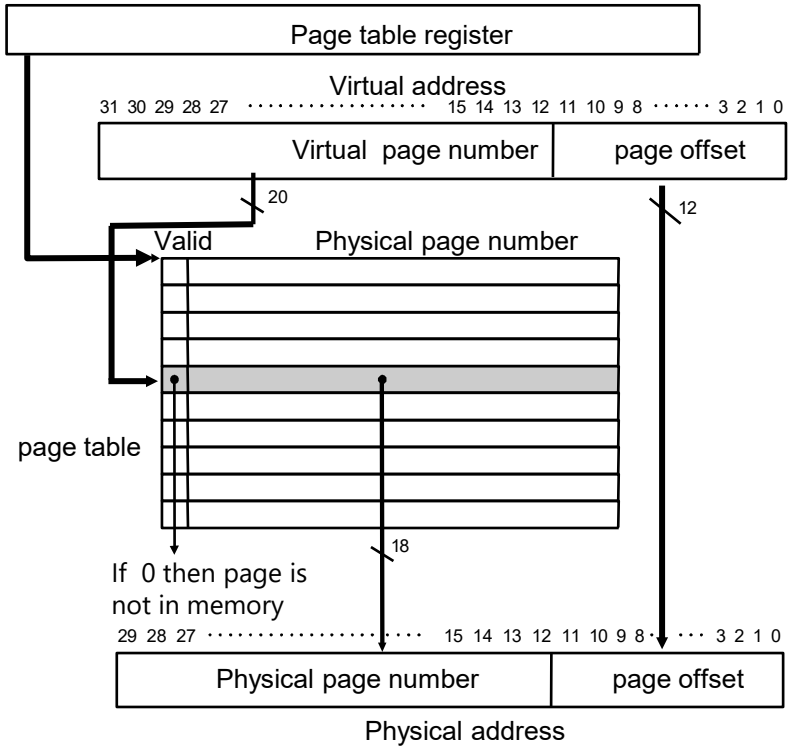


- Note: no PID needed since all virtual pages belong to one process
- **Pro**: Indexing can be done in constant time (**low time complexity**)
- **Con**: Page table as big as virtual mem (**high space complexity**)

Regular Page Table: Translation is super fast

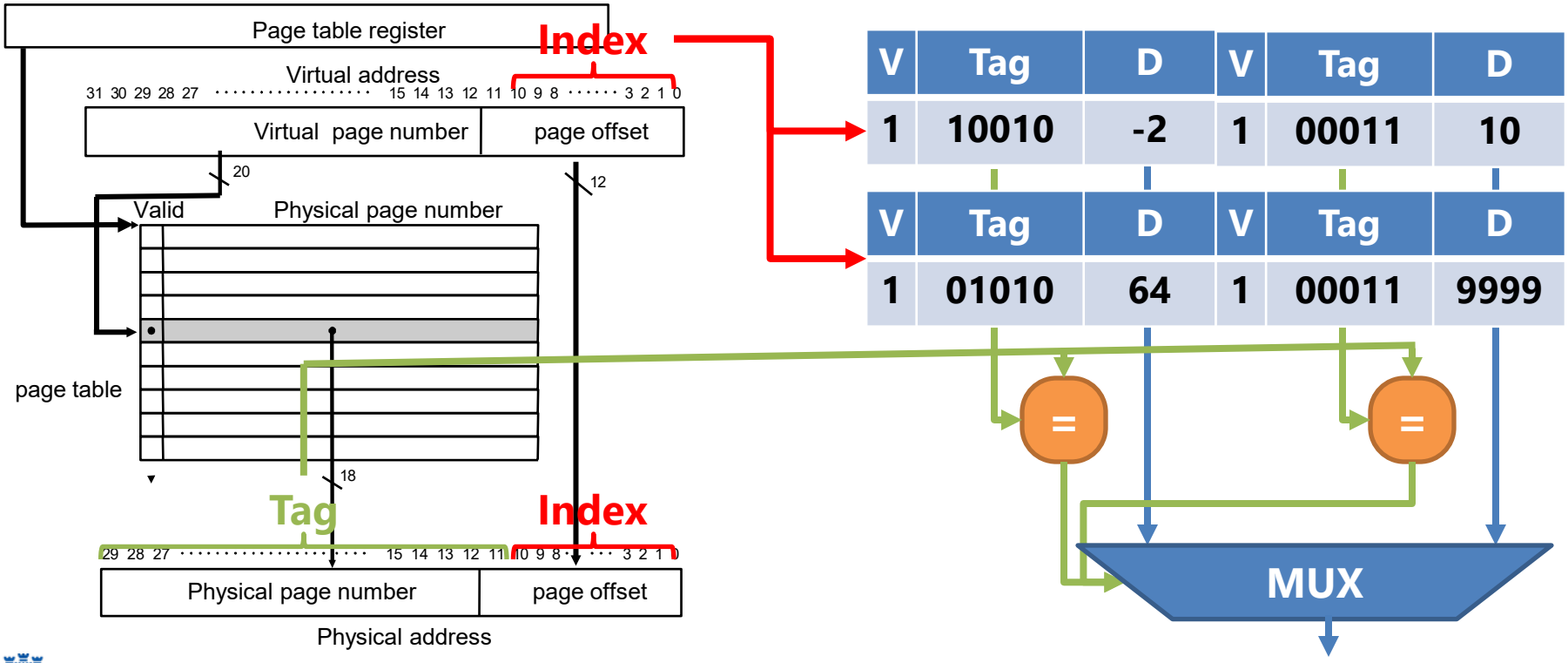
1. CPU extracts virtual page number from virtual address
2. CPU locates page table pointed to by **page table register**
3. Page table is indexed using virtual page number





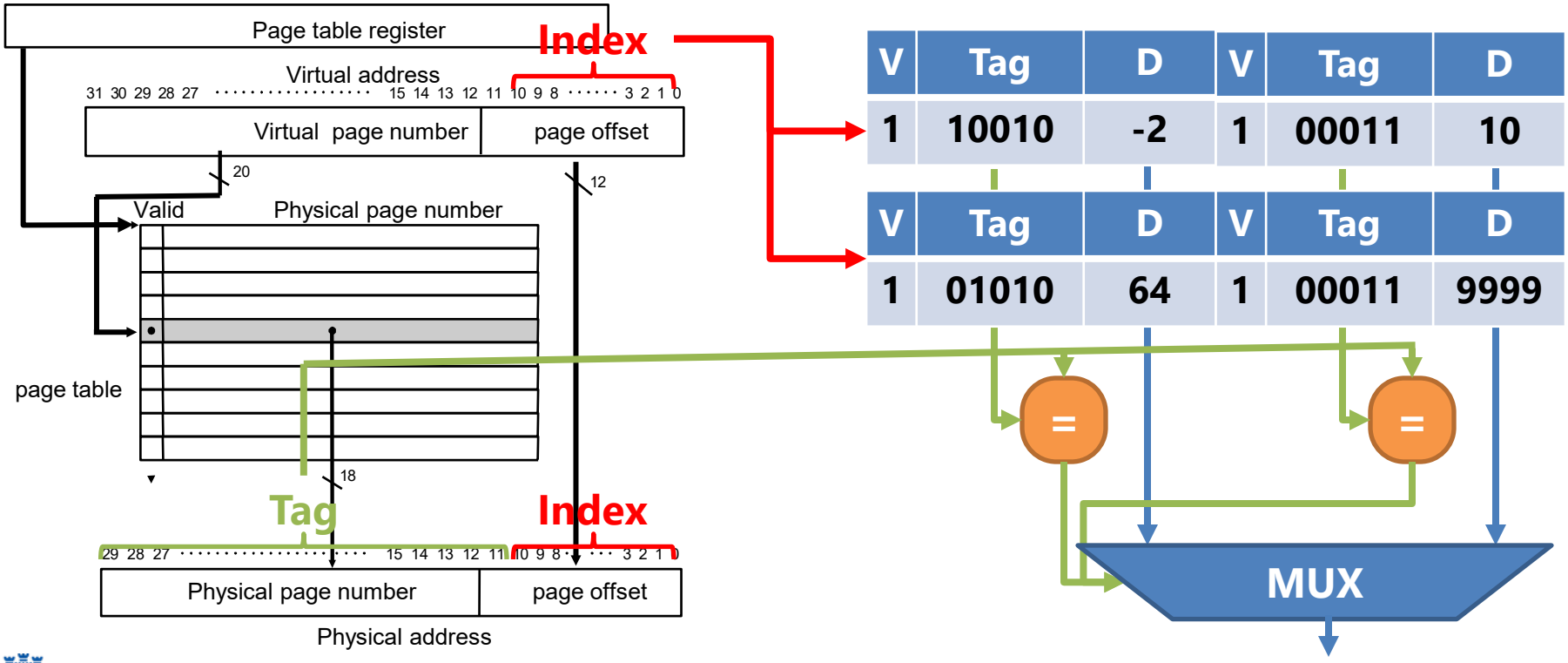
Regular Page Table: Translation delay can be hidden

- If index bits of cache does not exceed page offset bits (12 bits)
  - Cache row wire + bit wire delay can be overlapped with translation
  - Only additional delay is tag comparator + mux to choose block



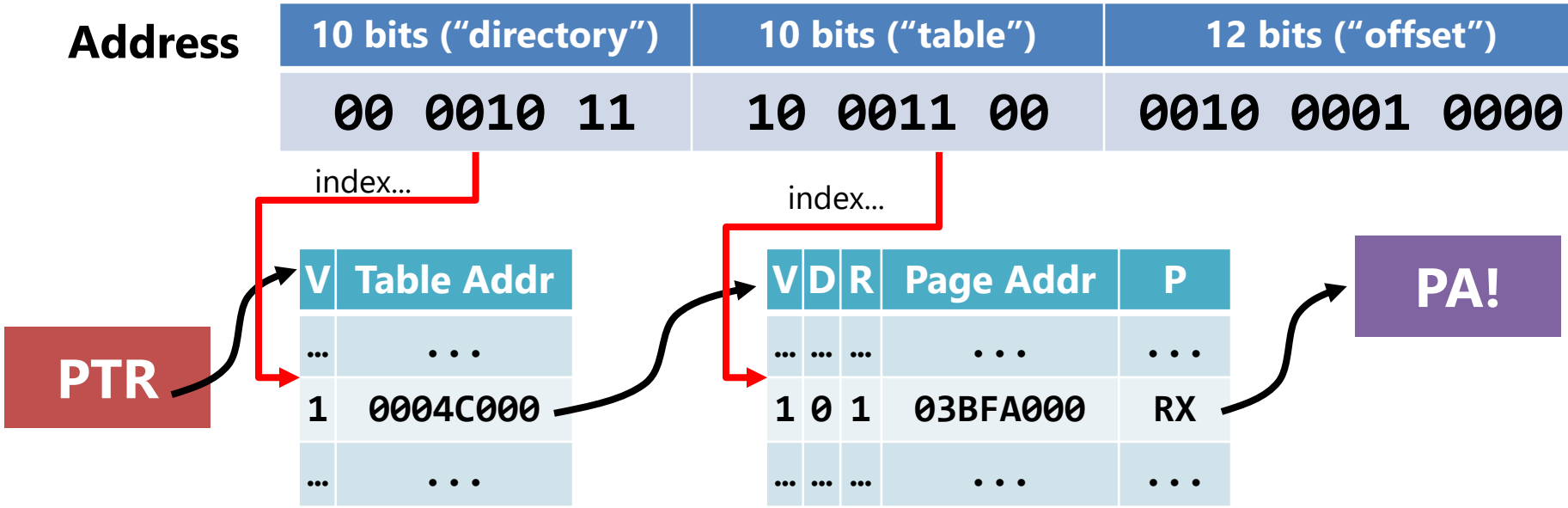
Regular Page Table: Translation delay can be hidden

- Limits the number of sets L1 cache can have, in order to be fast
  - E.g. thompson.cs.pitt.edu: L1 cache is 32 KB 8-way set associative
  - 32 KB / 8 = 4 KB per way = Can be indexed with exactly 12 bits



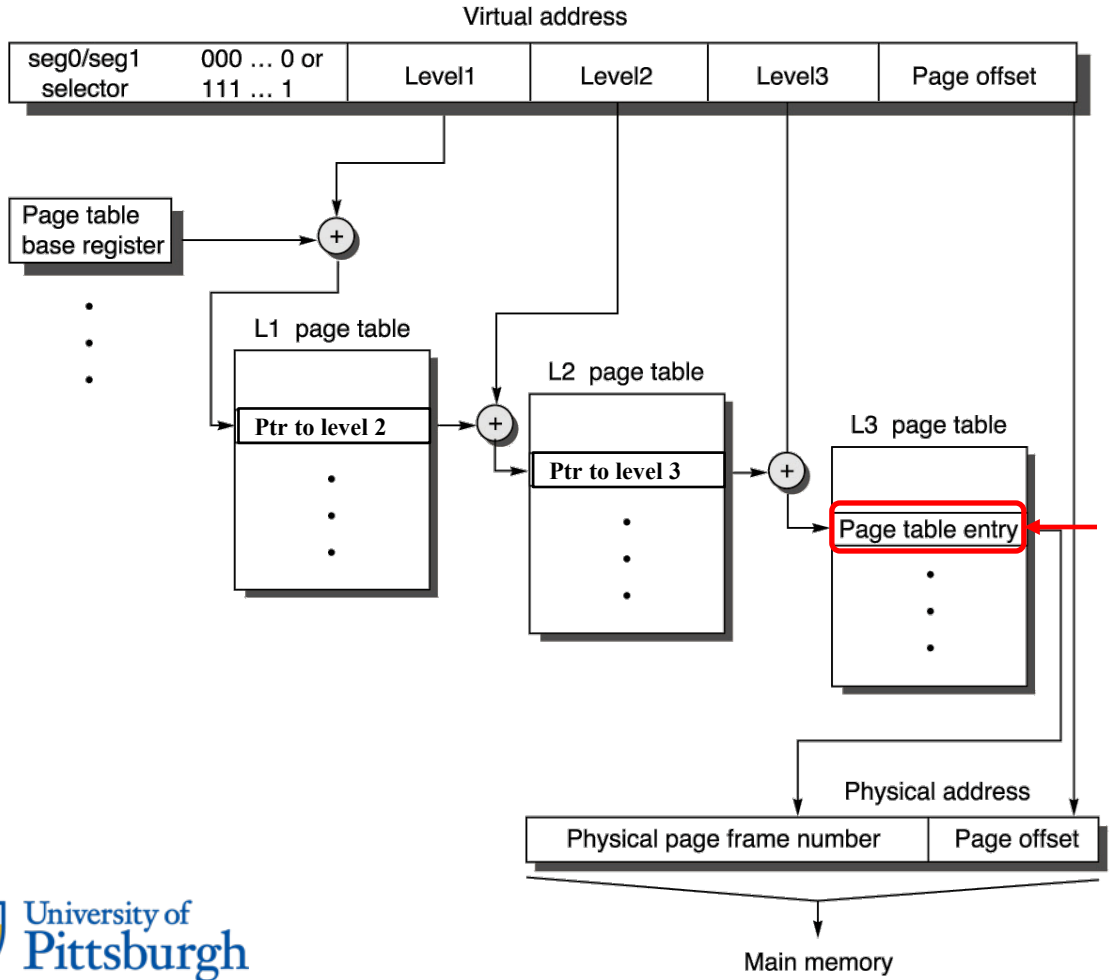
But how about the high space complexity?

- 32-bit addresses with 4KiB (2^{12} B) pages means **2^{20}** (**1M**) PTEs.
- 64-bit addresses with 4KiB pages means **2^{52}** (**4000 Tera**) PTEs.
- A **sparse data structure** makes sense (most PTEs are invalid)
  - **Hierarchical page tables** are a popular sparse data structure



CPU needs a "page table walker" to go get to PTE

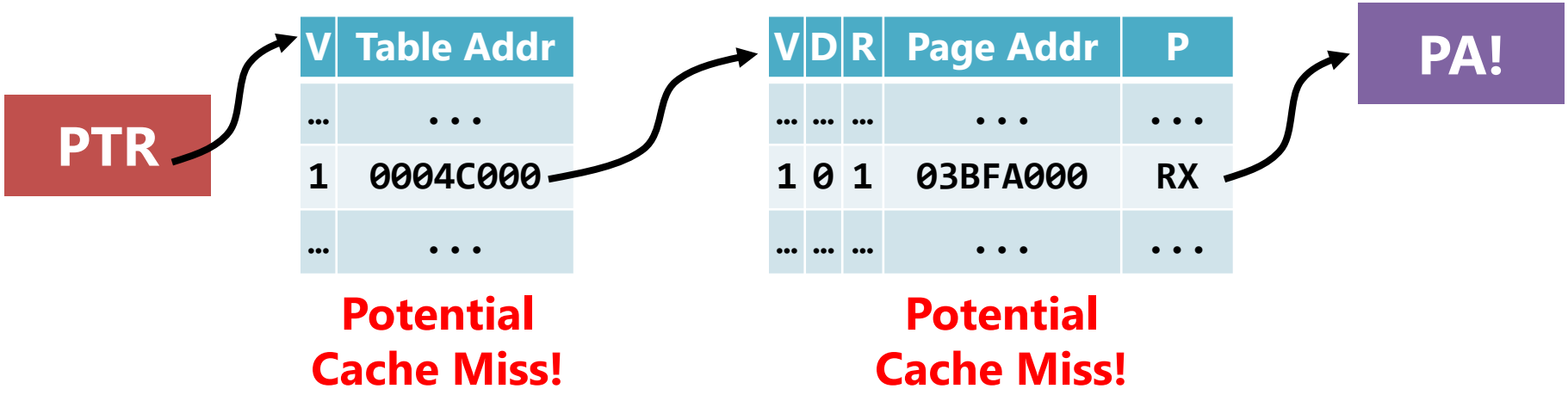
- Alpha 21264 CPU with 3-level page table:



At the end of the long walk,
the PTE (Page Table Entry) is all you need

Page table walk can even suffer cache misses

- Let's say we do memory access on a two-level page table:

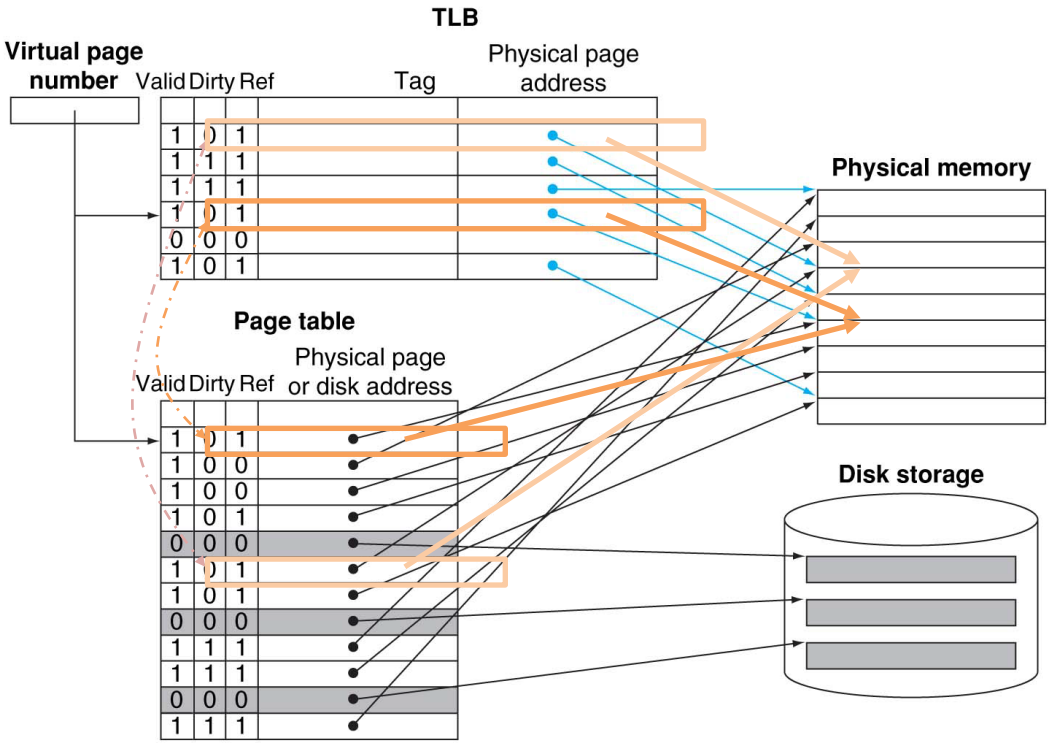


- Must perform as many memory accesses as there are levels
  - Cache misses can further delay page walk!

The TLB:
A Cache for Page Tables

TLB (Translation Lookaside Buffer) is a cache for PTEs

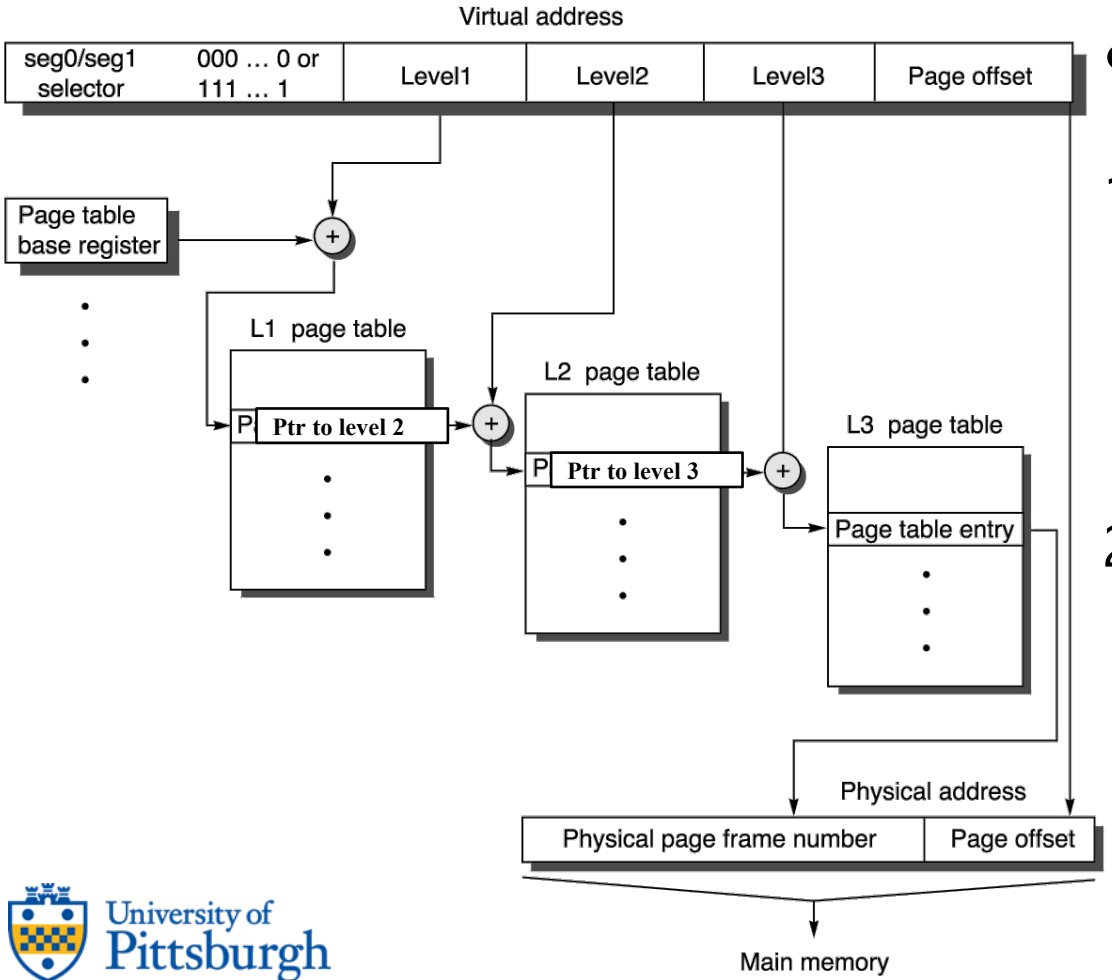
- **TLB**: A **cache** that contains frequently accessed **page table entries**



- TLB just like other caches resides within the CPU
- On a TLB **hit**:
  - No need to access page table in memory
- On a TBL **miss**:
  - Load PTE from page table
  - That means "walking" the hierarchical page table

On TLB miss, Page Table Walking needs to happen

- On a TLB miss, the CPU must "walk" the page table:



- Two options:

1. Software option
   - Miss raises OS exception
   - OS exception handler fills the TLB with PTE

2. Hardware option
   - CPU has special circuitry to walk page table (the **page table walker**)

   → Faster than SW option

TLB caches Page Table Entries (PTEs)

- The TLB holds PTEs – mappings from virtual to physical pages

Virtual Page Number →

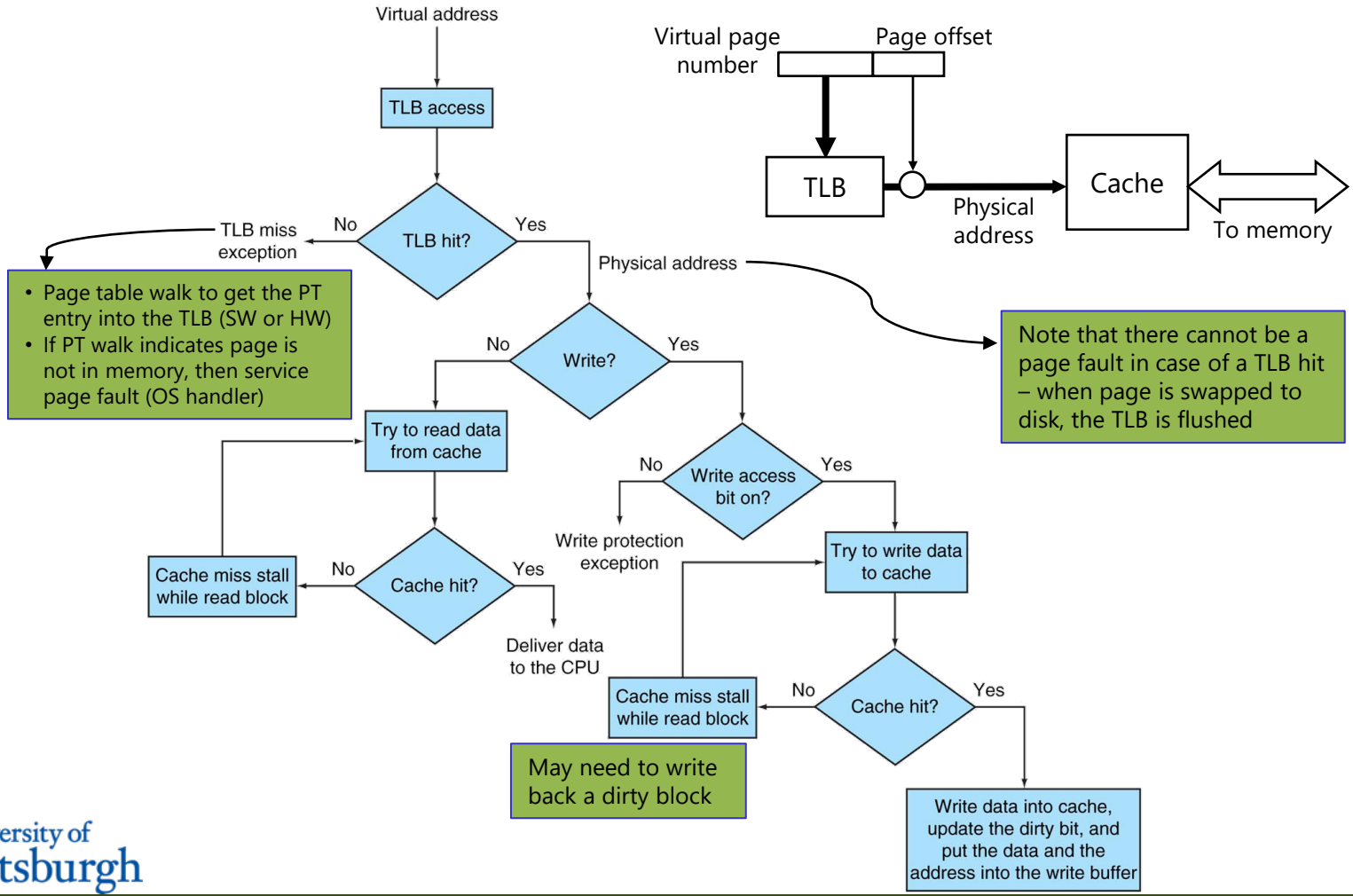
V	Virtual Page Number (Tag)	Permission bits	Physical Page Number
1	00008	RX	
1	FFFF3	RW	
…	…	…	

→ Physical Page Number

- Typically, a fully associate or set-associative cache
  - Since a TLB miss requires a page walk which is expensive!

- V == 1 && Virtual Page Number (Tag) Match → TLB hit
  - If access type is incompatible with permission bits, exception raised
  - Otherwise, Physical Page Number is returned

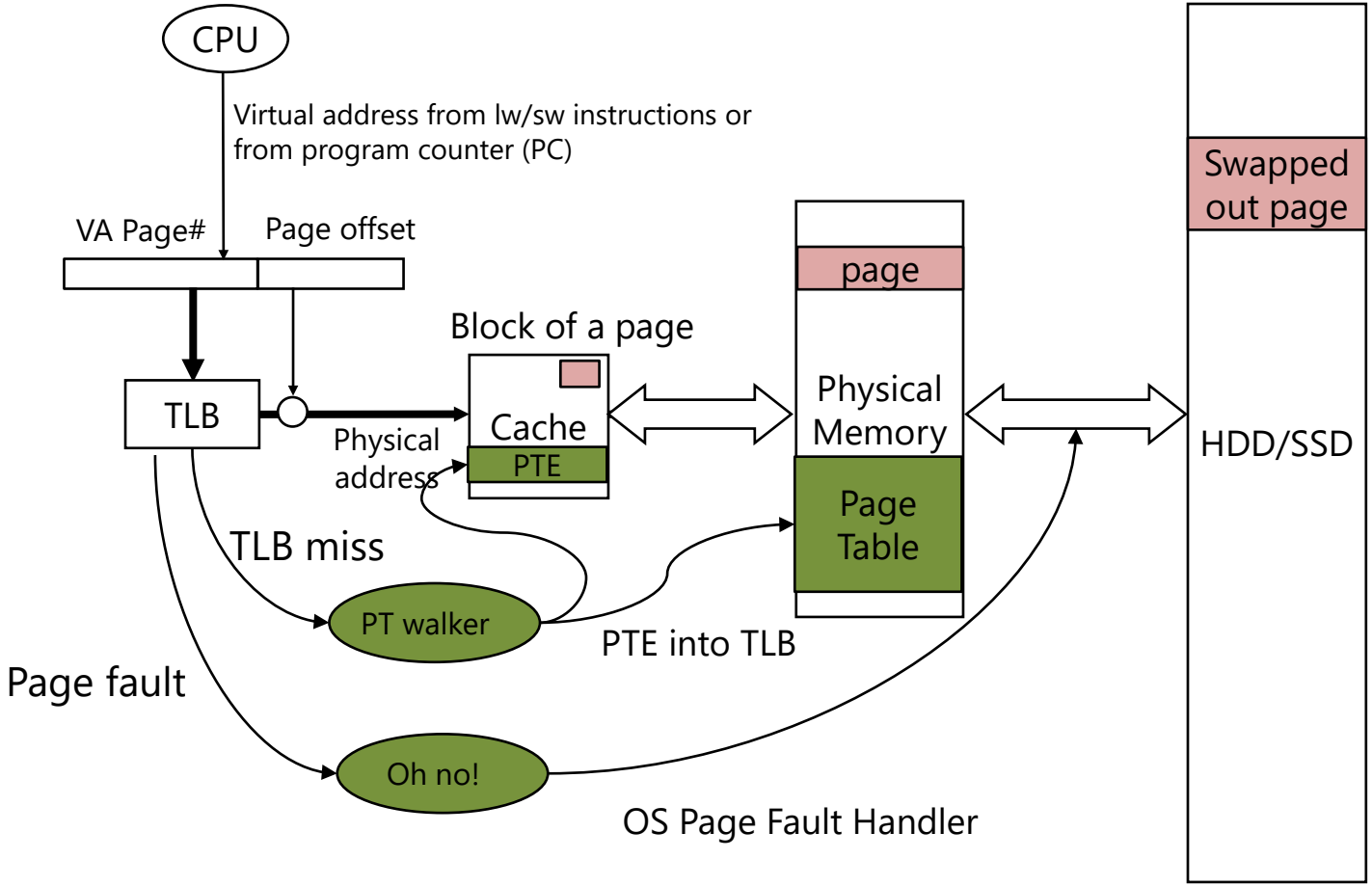
Memory Access Flowchart

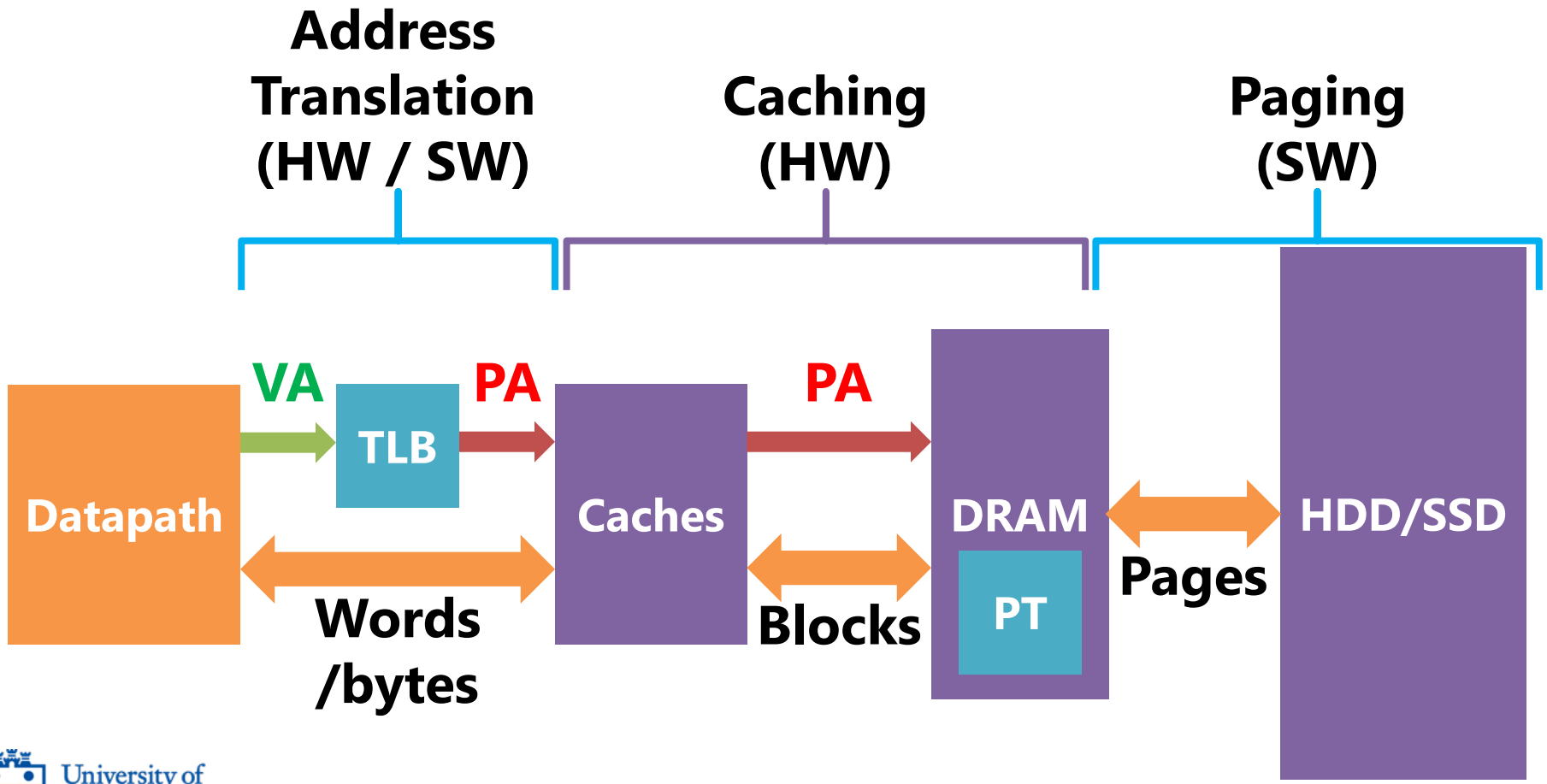


TLBs in Real Processors

Characteristic	ARM Cortex-A8	Intel Core i7
Virtual address	32 bits	48 bits
Physical address	32 bits	44 bits
Page size	Variable: 4, 16, 64 KiB, 1, 16 MiB	Variable: 4 KiB, 2/4 MiB
TLB organization	1 TLB for instructions and 1 TLB for data Both TLBs are fully associative, with 32 entries, round robin replacement TLB misses handled in hardware	1 TLB for instructions and 1 TLB for data per core Both L1 TLBs are four-way set associative, LRU replacement L1 I-TLB has 128 entries for small pages, 7 per thread for large pages L1 D-TLB has 64 entries for small pages, 32 for large pages The L2 TLB is four-way set associative, LRU replacement The L2 TLB has 512 entries TLB misses handled in hardware

Page table caching makes page table walks faster



Overall Memory System Design

- Fast memory access is possible through SW / HW collaboration: